

МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ
КРЕМЕНЧУЦЬКИЙ НАЦІОНАЛЬНИЙ УНІВЕРСИТЕТ
ІМЕНІ МИХАЙЛА ОСТРОГРАДСЬКОГО
КАФЕДРА АВТОМАТИЗАЦІЇ ТА ІНФОРМАЦІЙНИХ СИСТЕМ

ЗВІТ

ЗВІТ З ЛАБОРАТОРНИХ РОБІТ
З НАВЧАЛЬНОЇ ДИСЦИПЛІНИ
**«СУЧАСНІ ТЕХНОЛОГІЇ КОНТЕЙНЕРИЗАЦІЇ, РОЗГОРТАННЯ ТА
ПОБУДОВИ МАСШТАБОВАНИХ ПРОГРАМНИХ СИСТЕМ»**

Виконав студент групи КН-23-1

Полинько Ігор Миколайович

Перевірив: викладач Юдіна А. Л.

Кременчук 2026

ЛАБОРАТОРНА РОБОТА № 2

Тема: Робота з Linux-контейнером на основі образу Ubuntu

Мета: Навчитися завантажувати образ Ubuntu, створювати та запускати контейнер в інтерактивному режимі, працювати з його файловою системою, організовувати взаємодію контейнерів у користувацькій bridge-мережі та аналізувати доступні мережеві служби.

Порядок виконання роботи:

1. Перевірити готовність Docker Desktop і поточний тип контейнерів.
2. Завантажити образ ubuntu:24.04 і дослідити його метадані.
3. Створити контейнер ubuntu-lab та виконати базові команди Linux.
4. Створити файли й каталоги у файловій системі контейнера.
5. Оновити індекс пакетів та встановити iproute2, iputils-ping, curl і nmap.
6. Дослідити мережеві налаштування Ubuntu-контейнера.
7. Створити користувацьку мережу lab-network.
8. Запустити контейнер Nginx з іменем web-server у цій мережі.
9. Підключити ubuntu-lab до lab-network і перевірити розпізнавання імен.
10. Перевірити веб сервер за допомогою ping і curl.
11. Дослідити його порти та служби за допомогою nmap.
12. Перевірити збереження змін після зупинення контейнера та їх втрату після його видалення.
13. Виконати індивідуальне завдання за своїм варіантом.
14. Оформити звіт і відповісти на контрольні питання.

Індивідуальне завдання:

Після виконання загальної частини відновіть навчальне середовище: створіть lab-network, запустіть web-server і новий Ubuntu-контейнер у цій мережі.

Виконайте завдання згідно свого варіанту. Для кожної команди поясніть призначення параметрів, наведіть результат і сформулюйте висновок про можливості утиліти.

Варіант 15. Класичні мережеві утиліти: net-tools.

Установіть пакет net-tools та порівняйте класичні команди з сучасними командами пакета iproute2.

Ubuntu-контейнер:

apt install -y net-tools

ifconfig

route -n

netstat -rn

netstat -tuln

arp -n

1. Порівняйте ifconfig з ip addr.
2. Порівняйте route -n або netstat -rn з ip route.
3. Поясніть параметри netstat -tuln.
4. Визначте, чи слухає якийсь процес TCP- або UDP-порт усередині Ubuntu-контейнера.
5. Порівняйте таблицю ARP з IP-адресою web-server.

Хід роботи:

1. Запуск Docker Desktop і перевірте його стан:

```
PS C:\WINDOWS\system32> docker --version
Docker version 29.8.1, build 4a63305
PS C:\WINDOWS\system32> docker info --format '{{.OSType}}'
linux
```

Рисунок 2.1 — Перевірка стану Docker

2. Завантаження образу Ubuntu 24.04:

```
PS C:\WINDOWS\system32> docker pull ubuntu:24.04
24.04: Pulling from library/ubuntu
a60ef8ec184e: Pull complete
2ea90c0e4faa: Download complete
Digest: sha256:a853f94d226358a79c740cfc7bce0c289748f3fe3488d921d038ccd752c61b60
Status: Downloaded newer image for ubuntu:24.04
docker.io/library/ubuntu:24.04
PS C:\WINDOWS\system32> docker image ls ubuntu
```

IMAGE	ID	DISK USAGE	CONTENT SIZE	EXTRA
ubuntu:24.04	a853f94d2263	119MB	31.7MB	

Info → U In Use

Рисунок 2.2 — Процес завантаження образу Ubuntu 24.04

```

PS C:\WINDOWS\system32> docker image inspect ubuntu:24.04
[
  {
    "Id": "sha256:a853f94d226358a79c740cfc7bce0c289748f3fe3488d921d038ccd752c61b60",
    "RepoTags": [
      "ubuntu:24.04"
    ],
    "RepoDigests": [
      "ubuntu@sha256:a853f94d226358a79c740cfc7bce0c289748f3fe3488d921d038ccd752c61b60"
    ],
    "Created": "2026-09-18T21:20:38.103397332Z",
    "Config": {
      "Env": [
        "PATH=/usr/local/sbin:/usr/local/bin:/usr/sbin:/usr/bin:/sbin:/bin"
      ],
      "Cmd": [
        "/bin/bash"
      ],
      "Labels": {
        "org.opencontainers.image.version": "24.04"
      }
    },
    "Architecture": "amd64",
    "Os": "linux",
    "Size": 117400818,
    "RootFS": {
      "Type": "layers",
      "Layers": [
        "sha256:d81fb2a7c6ee3be05fdefaa7fd223be219c1ab172083ec1b994c3ecda33ac0a8"
      ]
    },
    "Metadata": {
      "LastTagTime": "2026-10-03T13:49:20.1660568Z"
    },
    "Descriptor": {
      "mediaType": "application/vnd.oci.image.index.v1+json",
      "digest": "sha256:a853f94d226358a79c740cfc7bce0c289748f3fe3488d921d038ccd752c61b60",
      "size": 6688
    },
    "Identity": {
      "Pull": [
        {
          "Repository": "docker.io/library/ubuntu"
        }
      ]
    }
  }
]

```

Рисунок 2.3 — Ідентифікатор образу, архітектура, операційна система та дата створення.

3-4. Створення і запуск контейнер Ubuntu, дослідження середовища

контейнера:

```
PS C:\WINDOWS\system32> docker run -it --name ubuntu-lab ubuntu:24.04 bash
root@1f1c7b66261f:/# whoami
root
root@1f1c7b66261f:/# pwd
/
root@1f1c7b66261f:/# hostname
1f1c7b66261f
root@1f1c7b66261f:/# uname -a
Linux 1f1c7b66261f 6.18.33.2-microsoft-standard-WSL2 #1 SMP PREEMPT_DYNAMIC Thu Jun 18 21:54:43 UTC 2026 x86_64 x86_6
4 x86_64 GNU/Linux
root@1f1c7b66261f:/# cat /etc/os-release
PRETTY_NAME="Ubuntu 24.04.5 LTS"
NAME="Ubuntu"
VERSION_ID="24.04"
VERSION="24.04.5 LTS (Noble Numbat)"
VERSION_CODENAME=noble
ID=ubuntu
ID_LIKE=debian
HOME_URL="https://www.ubuntu.com/"
SUPPORT_URL="https://help.ubuntu.com/"
BUG_REPORT_URL="https://bugs.launchpad.net/ubuntu/"
PRIVACY_POLICY_URL="https://www.ubuntu.com/legal/terms-and-policies/privacy-policy"
UBUNTU_CODENAME=noble
LOGO=ubuntu-logo
root@1f1c7b66261f:/# ls /
bin boot dev etc home lib lib64 media mnt opt proc root run sbin srv sys tmp usr var
root@1f1c7b66261f:/# ls -la/
ls: invalid option -- '/'
Try 'ls --help' for more information.
root@1f1c7b66261f:/# ls -la /
total 56
drwxr-xr-x  1 root root 4096 Oct  3 14:00 .
drwxr-xr-x  1 root root 4096 Oct  3 14:00 ..
-rwxr-xr-x  1 root root   0 Oct  3 14:00 .dockerenv
lrwxrwxrwx  1 root root   7 Apr 22  2024 bin -> usr/bin
drwxr-xr-x  2 root root 4096 Apr 22  2024 boot
drwxr-xr-x  5 root root 360 Oct  3 14:00 dev
drwxr-xr-x  1 root root 4096 Oct  3 14:00 etc
drwxr-xr-x  3 root root 4096 Sep 17 02:29 home
lrwxrwxrwx  1 root root   7 Apr 22  2024 lib -> usr/lib
lrwxrwxrwx  1 root root   9 Apr 22  2024 lib64 -> usr/lib64
drwxr-xr-x  2 root root 4096 Sep 17 02:20 media
drwxr-xr-x  2 root root 4096 Sep 17 02:20 mnt
drwxr-xr-x  2 root root 4096 Sep 17 02:20 opt
dr-xr-xr-x 263 root root   0 Oct  3 14:00 proc
drwx----- 2 root root 4096 Sep 17 02:29 root
drwxr-xr-x  4 root root 4096 Sep 17 02:29 run
lrwxrwxrwx  1 root root   8 Apr 22  2024 sbin -> usr/sbin
drwxr-xr-x  2 root root 4096 Sep 17 02:20 srv
dr-xr-xr-x 13 root root   0 Oct  3 14:00 sys
drwxrwxrwt  2 root root 4096 Sep 17 02:29 tmp
drwxr-xr-x 12 root root 4096 Sep 17 02:20 usr
drwxr-xr-x 11 root root 4096 Sep 17 02:29 var
root@1f1c7b66261f:/#
```

Рисунок 2.4 — Контейнер Ubuntu, середовище контейнера

5. Виконання операції з каталогами та файлами:

```
root@1f1c7b66261f:/# mkdir /lab
root@1f1c7b66261f:/# cd /lab
root@1f1c7b66261f:/lab# touch test.txt
root@1f1c7b66261f:/lab# echo 'Hello from Ubuntu container'>test.txt
root@1f1c7b66261f:/lab# cat test.txt
Hello from Ubuntu container
root@1f1c7b66261f:/lab# cp test.txt copy.txt
root@1f1c7b66261f:/lab# mv copy.txt result.txt
root@1f1c7b66261f:/lab# ls -la
total 16
drwxr-xr-x 2 root root 4096 Oct  3 14:12 .
drwxr-xr-x 1 root root 4096 Oct  3 14:11 ..
-rw-r--r-- 1 root root  28 Oct  3 14:12 result.txt
-rw-r--r-- 1 root root  28 Oct  3 14:11 test.txt
root@1f1c7b66261f:/lab# rm result.txt
root@1f1c7b66261f:/lab# ls -la
total 12
drwxr-xr-x 2 root root 4096 Oct  3 14:12 .
drwxr-xr-x 1 root root 4096 Oct  3 14:11 ..
-rw-r--r-- 1 root root  28 Oct  3 14:11 test.txt
```

Рисунок 2.5 — Операції з каталогами та файлами

6. Оновлення індексів пакетів і встановлення утиліт:

```
Setting up openssl (3.0.13-0ubuntu3.16) ...
Setting up libpam-cap:amd64 (1:2.66-5ubuntu2.4) ...
debconf: unable to initialize frontend: Dialog
debconf: (No usable dialog-like program is installed, so the dialog based frontend cannot be used. at /usr/share/perl5/Debconf/FrontEnd/Dialog.pm line 79.)
debconf: falling back to frontend: Readline
debconf: unable to initialize frontend: Readline
debconf: (Can't locate Term/ReadLine.pm in @INC (you may need to install the Term::ReadLine module) (@INC entries checked: /etc/perl /usr/local/lib/x86_64-linux-gnu/perl/5.38.2 /usr/local/share/perl/5.38.2 /usr/lib/x86_64-linux-gnu/perl5/5.38 /usr/share/perl5 /usr/lib/x86_64-linux-gnu/perl-base /usr/lib/x86_64-linux-gnu/perl/5.38 /usr/share/perl/5.38 /usr/local/lib/site_perl) at /usr/share/perl5/Debconf/FrontEnd/Readline.pm line 8.)
debconf: falling back to frontend: Teletype
Setting up iputils-ping (3:20240117-1ubuntu0.1) ...
Setting up publicsuffix (20231001.0357-0.1) ...
Setting up libldap2:amd64 (2.6.10+dfsg-0ubuntu0.24.04.1) ...
Setting up dbus-bin (1.14.10-4ubuntu4.1) ...
Setting up libbpf1:amd64 (1:1.3.0-2build2) ...
Setting up iproute2 (6.1.0-1ubuntu6.4) ...
debconf: unable to initialize frontend: Dialog
debconf: (No usable dialog-like program is installed, so the dialog based frontend cannot be used. at /usr/share/perl5/Debconf/FrontEnd/Dialog.pm line 79.)
debconf: falling back to frontend: Readline
debconf: unable to initialize frontend: Readline
debconf: (Can't locate Term/ReadLine.pm in @INC (you may need to install the Term::ReadLine module) (@INC entries checked: /etc/perl /usr/local/lib/x86_64-linux-gnu/perl/5.38.2 /usr/local/share/perl/5.38.2 /usr/lib/x86_64-linux-gnu/perl5/5.38 /usr/share/perl5 /usr/lib/x86_64-linux-gnu/perl-base /usr/lib/x86_64-linux-gnu/perl/5.38 /usr/share/perl/5.38 /usr/local/lib/site_perl) at /usr/share/perl5/Debconf/FrontEnd/Readline.pm line 8.)
debconf: falling back to frontend: Teletype
Setting up dbus-daemon (1.14.10-4ubuntu4.1) ...
Setting up liblinear4:amd64 (2.3.0+dfsg-5build1) ...
Setting up libnl-route-3-200:amd64 (3.7.0-0.3build1.1) ...
Setting up ca-certificates (20260601~24.04.1) ...
debconf: unable to initialize frontend: Dialog
debconf: (No usable dialog-like program is installed, so the dialog based frontend cannot be used. at /usr/share/perl5/Debconf/FrontEnd/Dialog.pm line 79.)
debconf: falling back to frontend: Readline
debconf: unable to initialize frontend: Readline
debconf: (Can't locate Term/ReadLine.pm in @INC (you may need to install the Term::ReadLine module) (@INC entries checked: /etc/perl /usr/local/lib/x86_64-linux-gnu/perl/5.38.2 /usr/local/share/perl/5.38.2 /usr/lib/x86_64-linux-gnu/perl5/5.38 /usr/share/perl5 /usr/lib/x86_64-linux-gnu/perl-base /usr/lib/x86_64-linux-gnu/perl/5.38 /usr/share/perl/5.38 /usr/local/lib/site_perl) at /usr/share/perl5/Debconf/FrontEnd/Readline.pm line 8.)
debconf: falling back to frontend: Teletype
Updating certificates in /etc/ssl/certs...
121 added, 0 removed; done.
Setting up dbus (1.14.10-4ubuntu4.1) ...
Setting up libgssapi-krb5-2:amd64 (1.20.1-6ubuntu2.10) ...
Setting up libssh-4:amd64 (0.10.6-2ubuntu0.5) ...
Setting up libibverbs1:amd64 (50.0-2ubuntu0.2) ...
Setting up libcurl4t64:amd64 (8.5.0-2ubuntu10.15) ...
Setting up ibverbs-providers:amd64 (50.0-2ubuntu0.2) ...
Setting up libpcap0.8t64:amd64 (1.10.4-4.1ubuntu3.1) ...
Setting up curl (8.5.0-2ubuntu10.15) ...
Setting up nmap (7.94+git20230807.3be01efb1+dfsg-3build2) ...
Processing triggers for libc-bin (2.39-0ubuntu8.9) ...
Processing triggers for ca-certificates (20260601~24.04.1) ...
Updating certificates in /etc/ssl/certs...
0 added, 0 removed; done.
Running hooks in /etc/ca-certificates/update.d...
done.
root@1f1c7b66261f:/lab#
```

Рисунок 2.6 — Процес оновлення індексів та встановлення утиліт

```

root@1f1c7b66261f:/lab# ip -V
ip utility, iproute2-6.1.0, libbpf 1.3.0
root@1f1c7b66261f:/lab# ping -V
ping from iputils 20240117
libcap: yes, IDN: yes, NLS: no, error.h: yes, getrandom(): yes, __fpending(): yes
root@1f1c7b66261f:/lab# curl --version
curl 8.5.0 (x86_64-pc-linux-gnu) libcurl/8.5.0 OpenSSL/3.0.13 zlib/1.3 brotli/1.1.0 zstd/1.5.5 libidn2/2.3.7 libpsl/0
.21.2 (+libidn2/2.3.7) libssh/0.10.6/openssl/zlib nghttp2/1.59.0 librtmp/2.3 OpenLDAP/2.6.10
Release-Date: 2023-12-06, security patched: 8.5.0-2ubuntu10.15
Protocols: dict file ftp ftps gopher gophers http https imap imaps ldap ldaps mqtt pop3 pop3s rtmp rtsp scp sftp smb
smbs smtp smtps telnet tftp
Features: alt-svc AsynchDNS brotli GSS-API HSTS HTTP2 HTTPS-proxy IDN IPv6 Kerberos Largefile libz NTLM PSL SPNEGO SS
L threadsafe TLS-SRP UnixSockets zstd
root@1f1c7b66261f:/lab# nmap --version
Nmap version 7.94SVN ( https://nmap.org )
Platform: x86_64-pc-linux-gnu
Compiled with: liblua-5.4.6 openssl-3.0.13 libssh2-1.11.0 libz-1.3 libpcap-1.10.4 nmap-libdnet-1.12 ip
v6
Compiled without:
Available nsock engines: epoll poll select

```

Рисунок 2.7 — Версія nmap

7. Дослідження мережевих параметрів Ubuntu-контейнера:

```

root@1f1c7b66261f:/lab# ip addr
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
        valid_lft forever preferred_lft forever
    inet6 ::1/128 scope host
        valid_lft forever preferred_lft forever
2: eth0@if5: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc noqueue state UP group default
    link/ether 0e:34:fb:08:13:93 brd ff:ff:ff:ff:ff:ff link-netnsid 0
    inet 172.17.0.2/16 brd 172.17.255.255 scope global eth0
        valid_lft forever preferred_lft forever
root@1f1c7b66261f:/lab# ip route
default via 172.17.0.1 dev eth0
172.17.0.0/16 dev eth0 proto kernel scope link src 172.17.0.2
root@1f1c7b66261f:/lab# hostname -I
172.17.0.2
root@1f1c7b66261f:/lab# cat /etc/resolv.conf
# Generated by Docker Engine.
# This file can be edited; Docker Engine will not make further changes once it
# has been modified.

nameserver 192.168.65.7

# Based on host file: '/etc/resolv.conf' (legacy)
# Overrides: []
root@1f1c7b66261f:/lab#

```

Рисунок 2.8 — Мережеві параметри Ubuntu-контейнера

8. Завершення Bash і перевірка стану контейнера

```

root@1f1c7b66261f:/lab# exit
exit
PS C:\WINDOWS\system32> docker ps
CONTAINER ID   IMAGE     COMMAND   CREATED   STATUS    PORTS     NAMES
PS C:\WINDOWS\system32> docker ps -a
CONTAINER ID   IMAGE     COMMAND   CREATED        STATUS      PORTS     NAMES
1f1c7b66261f   ubuntu:24.04   "bash"    38 minutes ago   Exited (0) 9 seconds ago         ubuntu-lab
PS C:\WINDOWS\system32>

```

Рисунок 2.9 — Завершення Bash і стан контейнера

9. Створіть користувацьку Docker-мережу:

```
PS C:\WINDOWS\system32> docker network create lab-network
bc050c36743d39a841777c7372ba430a690fe67c5f0c3bfd8cc53ccfc378f6d3
PS C:\WINDOWS\system32> docker network ls
NETWORK ID        NAME                DRIVER             SCOPE
9054f7bcc9d2      bridge             bridge            local
27af06b41764      host               host              local
bc050c36743d      lab-network        bridge            local
01984b0e5799      none              null              local
PS C:\WINDOWS\system32> docker network inspect lab-network
[
  {
    "Name": "lab-network",
    "Id": "bc050c36743d39a841777c7372ba430a690fe67c5f0c3bfd8cc53ccfc378f6d3",
    "Created": "2026-10-03T15:02:34.856498221Z",
    "Scope": "local",
    "Driver": "bridge",
    "EnableIPv4": true,
    "EnableIPv6": false,
    "IPAM": {
      "Driver": "default",
      "Options": {},
      "Config": [
        {
          "Subnet": "172.18.0.0/16",
          "Gateway": "172.18.0.1"
        }
      ]
    },
    "Internal": false,
    "Attachable": false,
    "Ingress": false,
    "ConfigFrom": {
      "Network": ""
    },
    "ConfigOnly": false,
    "Options": {
      "com.docker.network.enable_ipv4": "true",
      "com.docker.network.enable_ipv6": "false"
    },
    "Labels": {},
    "Containers": {},
    "Status": {
      "IPAM": {
        "Subnets": {
          "172.18.0.0/16": {
            "IPsInUse": 3,
            "DynamicIPsAvailable": 65533
          }
        }
      }
    }
  }
]
```

Рисунок 2.10 — Створення користувацької Docker-мережі

10. Запуск контейнера із вебсервером Nginx:

```
PS C:\WINDOWS\system32> docker pull nginx
Using default tag: latest
latest: Pulling from library/nginx
46243d3234ed: Pull complete
2056b40bae09: Pull complete
6b37362b3da7: Pull complete
f802f27d954b: Pull complete
f1169c633cbc: Pull complete
3326c3817340: Pull complete
afa8dec48454: Pull complete
e40088050cb6: Download complete
37d8c7707e42: Download complete
Digest: sha256:abe47724e466aeab9a345d8e46a221c2fa8953c7848bb4a3bd9976a7199f8cf2
Status: Downloaded newer image for nginx:latest
docker.io/library/nginx:latest
PS C:\WINDOWS\system32> docker run -d --name web-server --network lab-network nginx
ee7322f62e98fe9650e1d649c270bea7fcc3988565865cb151e75f997b40901d
PS C:\WINDOWS\system32> docker ps
```

CONTAINER ID	IMAGE	COMMAND	CREATED	STATUS	PORTS	NAMES
ee7322f62e98	nginx	"/docker-entrypoint..."	13 seconds ago	Up 12 seconds	80/tcp	web-server

Рисунок 2.11 — Запуск контейнера із вебсервером Nginx

```
"Volumes": null,
"WorkingDir": "",
"Entrypoint": [
  "/docker-entrypoint.sh"
],
"Labels": {
  "maintainer": "NGINX Docker Maintainers <u003cdocker-maint@nginx.com>u003e"
},
"StopSignal": "SIGQUIT"
},
"NetworkSettings": {
  "SandboxID": "96bad2605f040458174329f197ebba1195a67c96f9a6b311ab553ba74874c193",
  "SandboxKey": "/var/run/docker/netns/96bad2605f04",
  "Ports": {
    "80/tcp": null
  },
  "Networks": {
    "lab-network": {
      "IPAMConfig": null,
      "Links": null,
      "Aliases": null,
      "DriverOpts": null,
      "GwPriority": 0,
      "NetworkID": "bc050c36743d39a841777c7372ba430a690fe67c5f0c3bfd8cc53ccfc378f6d3",
      "EndpointID": "d6d4d9288e27ccb0fbc9a1ebd20677889c3317501d135ce65ea45b555e915db2",
      "Gateway": "172.18.0.1",
      "IPAddress": "172.18.0.2",
      "MacAddress": "2e:df:08:c5:55:2f",
      "IPPrefixLen": 16,
      "IPv6Gateway": "",
      "GlobalIPv6Address": "",
      "GlobalIPv6PrefixLen": 0,
      "DNSNames": [
        "web-server",
        "ee7322f62e98"
      ]
    }
  }
},
"ImageManifestDescriptor": {
  "mediaType": "application/vnd.oci.image.manifest.v1+json",
  "digest": "sha256:9c0f39aa1c46f5062ba080555a067ebb82d5f63a8cf4d63f6ef2e76fc9ff2d0c",
  "size": 2290,
  "annotations": {
    "com.docker.official-images.bashbrew.arch": "amd64",
    "org.opencontainers.image.base.digest": "sha256:7792b1f7702a86946cd518db72b6a407302c3e9bc1635634368b878189e8221c",
    "org.opencontainers.image.base.name": "debian:trixie-slim",
    "org.opencontainers.image.created": "2026-09-19T00:19:58Z",
    "org.opencontainers.image.revision": "a16f1329e13e7273c4103f75d863ca625b75109e",
    "org.opencontainers.image.source": "https://github.com/nginx/docker-nginx.git#a16f1329e13e7273c4103f75d863ca625b75109e:mainline/debian",
    "org.opencontainers.image.url": "https://hub.docker.com/_/nginx",
    "org.opencontainers.image.version": "1.31.6"
  },
  "platform": {
    "architecture": "amd64",
    "os": "linux"
  }
}
}
```

Рисунок 2.12 — IP-адреса контейнера в мережі lab-network.

11. Повторний запуск Ubuntu-контейнера і підключення його до мережі

PowerShell:

```
PS C:\WINDOWS\system32> docker start ubuntu-lab
ubuntu-lab
PS C:\WINDOWS\system32> docker network connect lab-network ubuntu-lab
PS C:\WINDOWS\system32> docker network inspect lab-network
docker: unknown command: docker network inspect

Usage:  docker network

Run 'docker network --help' for more information
PS C:\WINDOWS\system32> docker network inspect lab-network
[
  {
    "Name": "lab-network",
    "Id": "bc050c36743d39a841777c7372ba430a690fe67c5f0c3bfd8cc53ccfc378f6d3",
    "Created": "2026-10-03T15:02:34.856498221Z",
    "Scope": "local",
    "Driver": "bridge",
    "EnableIPv4": true,
    "EnableIPv6": false,
    "IPAM": {
      "Driver": "default",
      "Options": {},
      "Config": [
        {
          "Subnet": "172.18.0.0/16",
          "Gateway": "172.18.0.1"
        }
      ]
    },
    "Internal": false,
    "Attachable": false,
    "Ingress": false,
    "ConfigFrom": {
      "Network": ""
    },
    "ConfigOnly": false,
    "Options": {
      "com.docker.network.enable_ipv4": "true",
      "com.docker.network.enable_ipv6": "false"
    },
    "Labels": {},
    "Containers": {
      "1f1c7b66261fefdd36f2aee25dfdf79830988f0597456f924a966c5b52dd77518": {
        "Name": "ubuntu-lab",
        "EndpointID": "63207abd0d87386423400cb9f8c77c379daffd5b51e05a7b03a1d882ced2acb1",
        "MacAddress": "9a:46:62:3f:98:7b",
        "IPv4Address": "172.18.0.3/16",
        "IPv6Address": ""
      },
      "ee7322f62e98fe9650e1d649c270bea7fcc3988565865cb151e75f997b40901d": {
        "Name": "web-server",
        "EndpointID": "d6d4d9288e27ccb0fbc9a1ebd20677889c3317501d135ce65ea45b555e915db2",
        "MacAddress": "2e:df:08:c5:55:2f",
        "IPv4Address": "172.18.0.2/16",
        "IPv6Address": ""
      }
    },
    "Status": {
      "IPAM": {
        "Subnets": {
          "172.18.0.0/16": {
            "IPsInUse": 5,
```

Рисунок 2.13 — Підключення контейнера до мережі

12. Перевірка мережевої взаємодії контейнерів:

```
PS C:\WINDOWS\system32> docker exec -it ubuntu-lab bash
root@1f1c7b66261f:/# ip addr
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
        valid_lft forever preferred_lft forever
    inet6 ::1/128 scope host
        valid_lft forever preferred_lft forever
2: eth0@if7: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc noqueue state UP group default
    link/ether 72:64:4b:a8:80:63 brd ff:ff:ff:ff:ff:ff link-netnsid 0
    inet 172.17.0.2/16 brd 172.17.255.255 scope global eth0
        valid_lft forever preferred_lft forever
3: eth1@if8: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc noqueue state UP group default
    link/ether 9a:46:62:3f:98:7b brd ff:ff:ff:ff:ff:ff link-netnsid 0
    inet 172.18.0.3/16 brd 172.18.255.255 scope global eth1
        valid_lft forever preferred_lft forever
root@1f1c7b66261f:/# ip route
default via 172.17.0.1 dev eth0
172.17.0.0/16 dev eth0 proto kernel scope link src 172.17.0.2
172.18.0.0/16 dev eth1 proto kernel scope link src 172.18.0.3
root@1f1c7b66261f:/# ping -c 4 web-server
PING web-server (172.18.0.2) 56(84) bytes of data.
64 bytes from web-server.lab-network (172.18.0.2): icmp_seq=1 ttl=64 time=0.115 ms
64 bytes from web-server.lab-network (172.18.0.2): icmp_seq=2 ttl=64 time=0.106 ms
64 bytes from web-server.lab-network (172.18.0.2): icmp_seq=3 ttl=64 time=0.049 ms
64 bytes from web-server.lab-network (172.18.0.2): icmp_seq=4 ttl=64 time=0.056 ms

--- web-server ping statistics ---
4 packets transmitted, 4 received, 0% packet loss, time 3035ms
rtt min/avg/max/mdev = 0.049/0.081/0.115/0.029 ms
root@1f1c7b66261f:/# getent hosts web-server
172.18.0.2      web-server
```

Рисунок 2.14 — Перевірка мережевої взаємодії контейнерів

```
root@1f1c7b66261f:/# curl http://web-server
<!DOCTYPE html>
<html>
<head>
<title>Welcome to nginx!</title>
<style>
html { color-scheme: light dark; }
body { width: 35em; margin: 0 auto;
font-family: Tahoma, Verdana, Arial, sans-serif; }
</style>
</head>
<body>
<h1>Welcome to nginx!</h1>
<p>If you see this page, nginx is successfully installed and working.
Further configuration is required for the web server, reverse proxy,
API gateway, load balancer, content cache, or other features.</p>

<p>For online documentation and support please refer to
<a href="https://nginx.org/">nginx.org</a>.<br>
To engage with the community please visit
<a href="https://community.nginx.org/">community.nginx.org</a>.<br>
For enterprise grade support, professional services, additional
security features and capabilities please refer to
<a href="https://f5.com/nginx">f5.com/nginx</a>.</p>

<p><em>Thank you for using nginx.</em></p>
</body>
</html>
```

Рисунок 2.15 — HTML-код стандартної сторінки Welcome to nginx!

13. Дослідження вебсервера за допомогою Nmap:

```
root@1f1c7b66261f:/# nmap web-server
Starting Nmap 7.94SVN ( https://nmap.org ) at 2026-10-03 15:22 UTC
Nmap scan report for web-server (172.18.0.2)
Host is up (0.0000060s latency).
rDNS record for 172.18.0.2: web-server.lab-network
Not shown: 999 closed tcp ports (reset)
PORT      STATE SERVICE
80/tcp    open  http
MAC Address: 2E:DF:08:C5:55:2F (Unknown)

Nmap done: 1 IP address (1 host up) scanned in 0.18 seconds
root@1f1c7b66261f:/# nmap -p 22.80.433 web-server
Starting Nmap 7.94SVN ( https://nmap.org ) at 2026-10-03 15:23 UTC
Error #487: Your port specifications are illegal.  Example of proper form: "-100,200-1024,T:3000-4000,U:60000-"
QUITTING!
root@1f1c7b66261f:/# nmap -sV web-server
Starting Nmap 7.94SVN ( https://nmap.org ) at 2026-10-03 15:23 UTC
Nmap scan report for web-server (172.18.0.2)
Host is up (0.0000050s latency).
rDNS record for 172.18.0.2: web-server.lab-network
Not shown: 999 closed tcp ports (reset)
PORT      STATE SERVICE VERSION
80/tcp    open  http      nginx 1.31.6
MAC Address: 2E:DF:08:C5:55:2F (Unknown)

Service detection performed. Please report any incorrect results at https://nmap.org/submit/ .
Nmap done: 1 IP address (1 host up) scanned in 6.32 seconds
root@1f1c7b66261f:/# nmap -Pn web-server
Starting Nmap 7.94SVN ( https://nmap.org ) at 2026-10-03 15:28 UTC
Nmap scan report for web-server (172.18.0.2)
Host is up (0.0000060s latency).
rDNS record for 172.18.0.2: web-server.lab-network
Not shown: 999 closed tcp ports (reset)
PORT      STATE SERVICE
80/tcp    open  http
MAC Address: 2E:DF:08:C5:55:2F (Unknown)

Nmap done: 1 IP address (1 host up) scanned in 0.13 seconds
root@1f1c7b66261f:/# nmap -Pn -p 80 -sV web-server
Starting Nmap 7.94SVN ( https://nmap.org ) at 2026-10-03 15:28 UTC
Nmap scan report for web-server (172.18.0.2)
Host is up (0.000044s latency).
rDNS record for 172.18.0.2: web-server.lab-network

PORT      STATE SERVICE VERSION
80/tcp    open  http      nginx 1.31.6
MAC Address: 2E:DF:08:C5:55:2F (Unknown)

Service detection performed. Please report any incorrect results at https://nmap.org/submit/ .
Nmap done: 1 IP address (1 host up) scanned in 6.24 seconds
```

Рисунок 2.16 — Дослідження вебсервера за допомогою Nmap

14. Перевірка збереження змін після зупинення контейнера

```
root@1f1c7b66261f:/# exit
exit

What's next:
  Try Docker Debug for seamless, persistent debugging tools in any container or image → docker debug ubuntu-lab
  Learn more at https://docs.docker.com/go/debug-cli/
PS C:\WINDOWS\system32> docker stop ubuntu-lab
ubuntu-lab
PS C:\WINDOWS\system32> docker start ubuntu-lab
ubuntu-lab
PS C:\WINDOWS\system32> docker exec -it ubuntu-lab bash
root@1f1c7b66261f:/# cat /lab/test/txt
cat: /lab/test/txt: No such file or directory
root@1f1c7b66261f:/# nmap --version
Nmap version 7.94SVN ( https://nmap.org )
Platform: x86_64-pc-linux-gnu
Compiled with: liblua-5.4.6 openssl-3.0.13 libssh2-1.11.0 libz-1.3 libpcap-1.0.4 nmap-libdnet-1.12 ip
v6
Compiled without:
Available nsock engines: epoll poll select
root@1f1c7b66261f:/# cat /lab/test.txt
Hello from Ubuntu container
```

Рисунок 2.17 — Збережені зміни після зупинення контейнера

15. Дослідження наслідків видалення контейнера:

```
root@1f1c7b66261f:/# exit
exit

What's next:
  Try Docker Debug for seamless, persistent debugging tools in any container or image → docker debug ubuntu-lab
  Learn more at https://docs.docker.com/go/debug-cli/
PS C:\WINDOWS\system32> docker stop ubuntu-lab
ubuntu-lab
PS C:\WINDOWS\system32> docker rm ubuntu-lab
ubuntu-lab
PS C:\WINDOWS\system32> docker run -it --name ubuntu-new ubuntu:24.04 bash
root@9760f4f7ceee:/# ls /lab
ls: cannot access '/lab': No such file or directory
root@9760f4f7ceee:/# nmap --version
bash: nmap: command not found
```

Рисунок 2.18 — Наслідки видалення контейнера

16. Завершення загальної частини:

```
root@9760f4f7ceee:/# exit
exit

What's next:
  Debug this container error with Gordon → docker ai "help me fix this container error"
PS C:\WINDOWS\system32> docker rm ubuntu-new
ubuntu-new
PS C:\WINDOWS\system32> docker stop web-server
web-server
PS C:\WINDOWS\system32> docker rm web-server
web-server
PS C:\WINDOWS\system32> docker network rm lab-network
lab-network
```

Рисунок 2.19 — Завершення загальної частини

Індивідуальне завдання:

```
PS C:\WINDOWS\system32> docker network create lab-network
d39b60e113a773b73f5f75ff67d006ba0ded33b313f38a7a62e5e8b761c562a8
PS C:\WINDOWS\system32> docker run -d --name web-server --network lab-network nginx
205fd08389d3a1ff1b037bb1af82fb558072880326039a811ec77f66832de9c6
PS C:\WINDOWS\system32> docker run -dit --name ubuntu-variant --network lab-network ubuntu:24.04 bash
652e2658ce26c3a88aeb98d6ecb96afec8bd0d3c9b2e160ad8bc534f6a0a1aae
PS C:\WINDOWS\system32> docker exec -it ubuntu-variant bash
root@652e2658ce26:/# apt update
Get:1 http://security.ubuntu.com/ubuntu noble-security InRelease [126 kB]
Get:2 http://archive.ubuntu.com/ubuntu noble InRelease [256 kB]
Get:3 http://security.ubuntu.com/ubuntu noble-security/restricted amd64 Packages [1943 kB]
Get:4 http://archive.ubuntu.com/ubuntu noble-updates InRelease [126 kB]
Get:5 http://archive.ubuntu.com/ubuntu noble-backports InRelease [126 kB]
Get:6 http://archive.ubuntu.com/ubuntu noble/main amd64 Packages [1808 kB]
Get:7 http://security.ubuntu.com/ubuntu noble-security/main amd64 Packages [1318 kB]
Get:8 http://security.ubuntu.com/ubuntu noble-security/multiverse amd64 Packages [50.0 kB]
Get:9 http://security.ubuntu.com/ubuntu noble-security/universe amd64 Packages [1543 kB]
Get:10 http://archive.ubuntu.com/ubuntu noble/restricted amd64 Packages [117 kB]
Get:11 http://archive.ubuntu.com/ubuntu noble/universe amd64 Packages [19.3 MB]
Get:12 http://archive.ubuntu.com/ubuntu noble/multiverse amd64 Packages [331 kB]
Get:13 http://archive.ubuntu.com/ubuntu noble-updates/restricted amd64 Packages [2127 kB]
Get:14 http://archive.ubuntu.com/ubuntu noble-updates/main amd64 Packages [1688 kB]
Get:15 http://archive.ubuntu.com/ubuntu noble-updates/universe amd64 Packages [2162 kB]
Get:16 http://archive.ubuntu.com/ubuntu noble-updates/multiverse amd64 Packages [56.2 kB]
Get:17 http://archive.ubuntu.com/ubuntu noble-backports/multiverse amd64 Packages [671 B]
Get:18 http://archive.ubuntu.com/ubuntu noble-backports/universe amd64 Packages [36.0 kB]
Get:19 http://archive.ubuntu.com/ubuntu noble-backports/main amd64 Packages [49.0 kB]
Fetched 33.2 MB in 4s (8780 kB/s)
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
3 packages can be upgraded. Run 'apt list --upgradable' to see them.
```

Рисунок 2.20 — Відновлення навчального середовища

```

root@652e2658ce26:/# apt install -y net-tools
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
The following NEW packages will be installed:
  net-tools
0 upgraded, 1 newly installed, 0 to remove and 3 not upgraded.
Need to get 204 kB of archives.
After this operation, 811 kB of additional disk space will be used.
Get:1 http://archive.ubuntu.com/ubuntu noble-updates/main amd64 net-tools amd64 2.10-0.1ubuntu4.4 [204 kB]
Fetched 204 kB in 1s (391 kB/s)
debconf: delaying package configuration, since apt-utils is not installed
Selecting previously unselected package net-tools.
(Reading database ... 4381 files and directories currently installed.)
Preparing to unpack .../net-tools_2.10-0.1ubuntu4.4_amd64.deb ...
Unpacking net-tools (2.10-0.1ubuntu4.4) ...
Setting up net-tools (2.10-0.1ubuntu4.4) ...
root@652e2658ce26:/# ifconfig
eth0: flags=4163<UP,BROADCAST,RUNNING,MULTICAST> mtu 1500
    inet 172.18.0.3 netmask 255.255.0.0 broadcast 172.18.255.255
    ether 52:40:64:c3:5f:f3 txqueuelen 0 (Ethernet)
    RX packets 36206 bytes 50908625 (50.9 MB)
    RX errors 0 dropped 0 overruns 0 frame 0
    TX packets 28298 bytes 2006696 (2.0 MB)
    TX errors 0 dropped 0 overruns 0 carrier 0 collisions 0

lo: flags=73<UP,LOOPBACK,RUNNING> mtu 65536
    inet 127.0.0.1 netmask 255.0.0.0
    inet6 ::1 prefixlen 128 scopeid 0x10<host>
    loop txqueuelen 1000 (Local Loopback)
    RX packets 6 bytes 818 (818.0 B)
    RX errors 0 dropped 0 overruns 0 frame 0
    TX packets 6 bytes 818 (818.0 B)
    TX errors 0 dropped 0 overruns 0 carrier 0 collisions 0

root@652e2658ce26:/# route -n
Kernel IP routing table
Destination Gateway Genmask Flags Metric Ref Use Iface
0.0.0.0 172.18.0.1 0.0.0.0 UG 0 0 0 eth0
172.18.0.0 0.0.0.0 255.255.0.0 U 0 0 0 eth0
root@652e2658ce26:/# netstat -rn
Kernel IP routing table
Destination Gateway Genmask Flags MSS Window irtt Iface
0.0.0.0 172.18.0.1 0.0.0.0 UG 0 0 0 eth0
172.18.0.0 0.0.0.0 255.255.0.0 U 0 0 0 eth0
root@652e2658ce26:/# netstat -tunl
Active Internet connections (only servers)
Proto Recv-Q Send-Q Local Address Foreign Address State
tcp 0 0 127.0.0.11:35979 0.0.0.0:* LISTEN
udp 0 0 127.0.0.11:56057 0.0.0.0:*
root@652e2658ce26:/# arp -n
Address HWtype HWaddress Flags Mask Iface
172.18.0.1 ether 92:c1:73:4f:1d:28 C eth0

```

Рисунок 2.21 — Установлення пакету net-tools та порівняння класичних команд із сучасними командами пакета iproute2.

1. Порівняння ifconfig з ip addr

Команди ifconfig та ip addr використовуються для перегляду мережевих інтерфейсів і їхніх параметрів. Команда ifconfig належить до класичного набору утиліт net-tools, тоді як ip addr є сучасним засобом пакета iproute2. Обидві команди дозволяють визначити назву інтерфейсу, його IP-адресу, MAC-адресу та стан. Команда ip addr є сучаснішою та має ширші можливості для керування мережевими параметрами.

2. Порівняння route -n та netstat -rn з ip route

Команди route -n і netstat -rn використовуються для перегляду таблиці маршрутизації. Сучасною альтернативою є команда ip route. Усі три команди дозволяють визначити мережі призначення, шлюз та інтерфейс, через який передаються пакети. Відмінність полягає в тому, що route і netstat належать до застарілого набору net-tools, а ip route є сучасним інструментом пакета iproute2.

3. Пояснення параметрів netstat -tuln

Команда netstat -tuln використовується для перегляду мережевих портів, які прослуховуються в контейнері. Параметр -t показує TCP-з'єднання, -u – UDP-з'єднання, -l – порти, які перебувають у стані прослуховування, а -n – відображає IP-адреси та номери портів у числовому вигляді без перетворення їх на імена. Таким чином, команда дозволяє визначити, які TCP- та UDP-порти доступні для приймання мережевих з'єднань.

4. Визначення процесів, які слухають TCP- або UDP-порти

Для визначення відкритих портів всередині контейнера було використано команду netstat -tuln. За її результатами можна встановити, чи є в Ubuntu-контейнері процеси, які прослуховують TCP- або UDP-порти. У базовому контейнері Ubuntu, де не запущено мережевих серверів, зазвичай відсутні порти, що прослуховуються. Вебсервер Nginx у цій роботі працює в окремому контейнері web-server, тому його порт 80 не повинен відображатися в netstat контейнера ubuntu-variant.

5. Порівняння таблиці ARP з IP-адресою web-server

За допомогою команди getent hosts web-server було визначено IP-адресу контейнера Nginx за його іменем. Команда arp -n дозволяє переглянути таблицю відповідності IP- та MAC-адрес у мережі. Порівняння цих даних дає змогу дослідити мережеву взаємодію між контейнерами. Наявність відповідного запису в ARP-таблиці залежить від того, чи відбувався нещодавно обмін мережевими пакетами. Ім'я web-server при цьому автоматично розпізнається Docker через вбудований DNS користувацької bridge-мережі.

Висновок:

На цій лабораторній роботі ми навчилися завантажувати образ Ubuntu, створювати та запускати контейнер в інтерактивному режимі, працювати з його файловою системою, організовувати взаємодію контейнерів у користувацькій bridge-мережі та аналізувати доступні мережеві служби.

Контрольні питання:

1. Чим образ Docker відрізняється від контейнера?

Образ Docker – це незмінний шаблон файлової системи, програм і налаштувань. Контейнер – це створений на основі образу запущений або зупинений екземпляр.

2. Навіщо в команді `docker run` використовується параметр `-it`?

Параметр `-i` залишає стандартний потік введення відкритим, а `-t` створює псевдотермінал. Разом `-it` дозволяють інтерактивно працювати з оболонкою контейнера через термінал.

3. Який процес є головним у контейнері `ubuntu-lab`?

Головним процесом є оболонка Bash, оскільки контейнер було запущено командою `bash`. Вона працює як процес PID 1.

4. Чому контейнер зупиняється після виконання `exit`?

Команда `exit` завершує головний процес Bash. Оскільки контейнер існує доти, доки працює його головний процес, після його завершення контейнер переходить у стан `Exited`.

5. Чим `docker run` відрізняється від `docker start`?

`docker run` створює новий контейнер на основі образу та запускає його. `docker start` запускає вже існуючий зупинений контейнер.

6. Чим `docker start` відрізняється від `docker exec`?

`docker start` запускає зупинений контейнер разом із його головним процесом. `docker exec` запускає додатковий процес усередині вже працюючого контейнера.

7. Для чого виконується `apt update` перед `apt install`?

`apt update` завантажує актуальну інформацію про доступні пакети та їхні версії з репозиторіїв. Після цього `apt install` може встановити потрібні пакети на основі оновленого індексу.

8. Чи змінюється початковий образ після встановлення ntar у контейнері?

Ні. Початковий образ ubuntu:24.04 не змінюється. Встановлений ntar записується у змінюваний шар конкретного контейнера.

9. Що відбудеться з файлами контейнера після docker stop?

Після docker stop контейнер зупиняється, але його файлові зміни та записуваний шар зберігаються. Після повторного запуску файли залишаються.

10. Що відбудеться з ними після docker rm?

docker rm видаляє контейнер разом із його записуваним шаром. Зміни, які не були збережені окремо, після цього втрачаються.

11. Що таке bridge-мережа Docker?

Bridge-мережа – це віртуальна мережа Docker, яка забезпечує мережеву взаємодію між підключеними контейнерами та ізолює їх від інших мереж.

12. Чим користувацька bridge-мережа відрізняється від стандартної bridge-мережі?

Користувацька bridge-мережа має автоматичне DNS-розпізнавання імен контейнерів. Тому контейнери можуть звертатися один до одного за іменами. Вона також дозволяє краще організовувати ізольовані групи контейнерів.

13. Як контейнери в користувацькій мережі знаходять один одного за іменами?

Docker надає користувацькій bridge-мережі вбудований DNS. Ім'я контейнера автоматично пов'язується з його IP-адресою, тому, наприклад, web-server можна використовувати замість його IP-адреси.

14. У чому різниця між IP-адресою контейнера й опублікованим портом хоста?

IP-адреса контейнера використовується для мережевої взаємодії всередині Docker-мережі. Опублікований порт хоста через параметр -p створює доступ до порту контейнера з боку Windows-хоста або зовнішньої мережі.

15. Для чого призначені ping, curl і nmap?

ping перевіряє доступність вузла та мережеве з'єднання. curl використовується для надсилання HTTP-запитів і отримання даних від вебсервера. nmap використовується для дослідження відкритих портів і мережевих служб.

16. Що означають стани порту open, closed і filtered у результатах nmap?

open означає, що на порту працює служба та приймає з'єднання. closed означає, що порт доступний, але служба на ньому не прослуховує з'єднання. filtered означає, що Nmap не може визначити стан порту через фільтрацію мережевим екраном або іншим механізмом.

17. Чому сканування nmap потрібно обмежувати системами, на дослідження яких отримано дозвіл?

Сканування мережевих портів є активною взаємодією з іншою системою та може розглядатися як несанкціоноване дослідження. Тому Nmap слід використовувати лише щодо власних систем або систем, на дослідження яких отримано відповідний дозвіл.

18. Чому uname -a та cat /etc/os-release описують різні складові середовища контейнера?

uname -a показує інформацію про ядро Linux, яке спільно використовується контейнерами та надається середовищем Docker Desktop/WSL 2. cat /etc/os-release показує інформацію про користувацьке середовище та дистрибутив Ubuntu всередині контейнера. Образ Ubuntu не містить власного ядра Linux.